

各式各样的 Self-attention 变型

从稀疏化、低秩压缩到 Linear Attention 与 Synthesizer



基于李宏毅老师《机器学习》课程整理

2026 年 6 月 8 日

视频信息

频道：李宏毅机器学习深度学习课程 (Bilibili)

时长：约 71 分钟

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=17>

目录

1 为什么要改造 Self-attention	4
1.1 改造 self-attention 的三条路线	5
1.2 本章小结	5
2 用人类知识决定哪些位置不用算	5
2.1 Local Attention 与 Truncated Attention	5
2.2 Global Attention: 给少数 token 全局权限	6
2.3 Longformer 与 BigBird: 固定稀疏图的组合	7
2.4 本章小结	8
3 让模型判断关键位置	9
3.1 只保留可能较大的 attention 值	9
3.2 Reformer 与 Routing Transformer: 用 clustering 筛连接	9
3.3 Sinkhorn Sorting Network: 可学习的稀疏 pattern	10
3.4 本章小结	11
4 低秩与代表性 Key/Value	12
4.1 Attention Matrix 可能是低秩的	12
4.2 减少 Key 与 Value 的数量	12
4.3 能不能减少 Query?	13
4.4 Compressed Attention 与 Linformer	14
4.5 本章小结	15
5 把 Attention 看成三矩阵乘法	15
5.1 乘法顺序为什么重要	16
5.2 本章小结	17
6 Linear Attention: 把 Softmax 拆成特征映射	17
6.1 Kernel 化的关键假设	17
6.2 分子重排: 从逐位置加权到全局模板	18
6.3 为什么它是线性的	19
6.4 不同论文如何实现	20
6.5 本章小结	21
7 还需要用 Q 和 K 算 Attention 吗	21
7.1 Synthesizer: Attention Matrix 可以是参数	22

7.2	Attention-free 与 MLP Mixing	22
7.3	本章小结	23
8	总结与延伸	23
8.1	一张方法地图	23
8.2	理解这些方法时最重要的三个问题	24
8.3	延伸阅读方向	24

1 为什么要改造 Self-attention

标准 self-attention 的优势是每一个位置都可以直接读取整个序列的信息。若输入序列为

$$A = (a^1, a^2, \dots, a^N),$$

模型会为每个位置产生 query、key 与 value:

$$q^i = W^q a^i, \quad k^i = W^k a^i, \quad v^i = W^v a^i.$$

接着每个 query 都要和每个 key 做相似度计算。若不考虑多头和缩放常数, 一个常见写法是

$$e_{ij} = q_i^\top k_j, \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{\ell=1}^N \exp(e_{i\ell})}, \quad b^i = \sum_{j=1}^N \alpha_{ij} v^j.$$

这里真正让人头痛的是 $\alpha \in \mathbb{R}^{N \times N}$ 。序列长度是 N , attention matrix 就有 N^2 个位置。对短句子来说这还可以接受; 对长文本、语音、图像 patch、蛋白质序列、长上下文语言模型来说, N^2 很快会成为主要瓶颈。

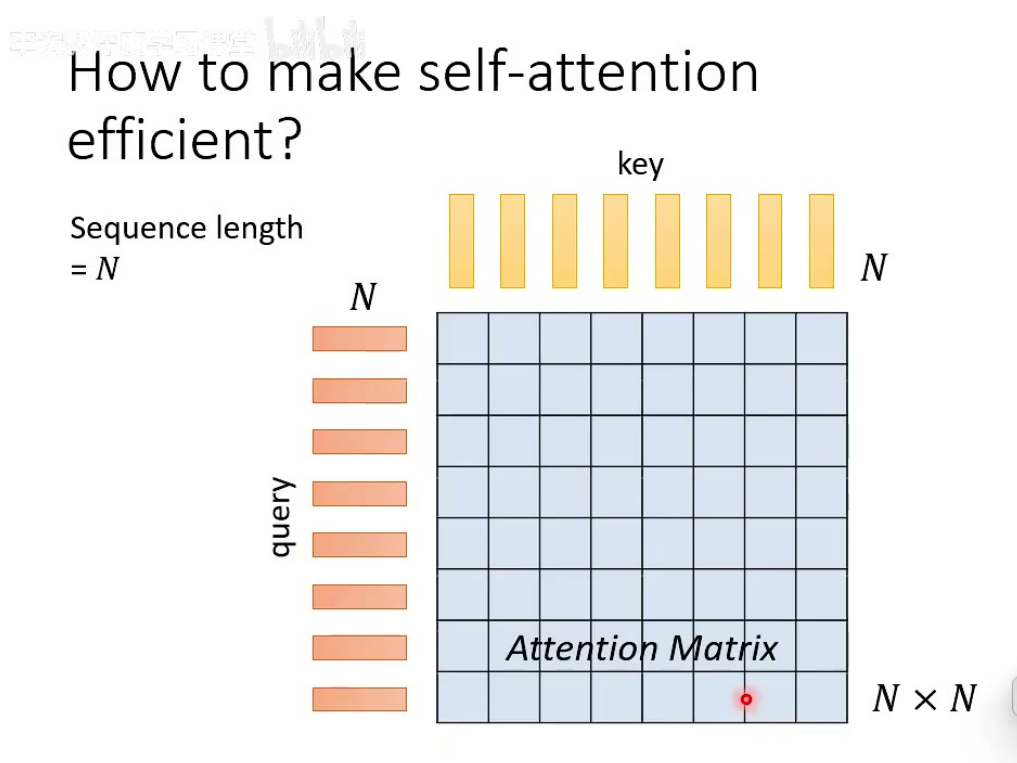


图 1: 标准 self-attention 需要产生 $N \times N$ 的 attention matrix; 每个 query 都要和每个 key 计算一次相似度, 因此长度一大, 矩阵本身就成为计算和记忆体瓶颈。²

复杂度可以粗略分成两块。第一块是产生 Q, K, V 的线性变换, 复杂度大约随 N 线性成长; 第二块是 $QK^\top$ 与 attention matrix 乘 value 的过程, 会出现 N^2 。当 N 不大时, 前者和后者未必差很多; 当 N 极大时, 后者通常会主导。

²视频画面时间区间: 约 02:15–03:05; 字幕附近说明「sequence 长度是 N 」「有 N 个 key、有 N 个 query」「得到一个 N 乘以 N 的矩阵」。

本讲的共同问题

这堂课讨论的各种方法看似五花八门，其实大多在回答同一个问题：我们能不能不要真的算完整的 $N \times N$ attention matrix？有的方法让矩阵变稀疏，有的方法让矩阵变瘦，有的方法完全改变乘法顺序，还有的方法甚至不再由 Q 和 K 产生 attention 权重。

1.1 改造 self-attention 的三条路线

从课程内容看，可以把方法分成三类。

第一类是**稀疏化**：保留 $N \times N$ 这个概念，但只计算其中一小部分位置。其余位置被视为零，或被 mask 掉。Local attention、Longformer、BigBird、Reformer、Routing Transformer、Sinkhorn Sorting Network 都可以放在这个大方向下理解。

第二类是**压缩矩阵大小**：不再用所有 key 或 value，而是选出代表性的 key/value，或者用线性组合把长序列压成短序列。Linformer 和 compressed attention 代表这类思路。

第三类是**改变公式**：把 attention 看成矩阵乘法，尝试改变乘法顺序，或者用特征映射 $\phi(\cdot)$ 近似 softmax kernel，从而绕开显式 $N \times N$ 矩阵。Linear Transformer、Efficient Attention、Performer 这类方法都属于这一支。

1.2 本章小结

Self-attention 的强大来自全局两两交互，瓶颈也来自全局两两交互。若输入长度为 N ，完整 attention matrix 有 N^2 个元素；本讲所有变型都在尝试用结构假设、数据相关估计、低秩压缩或代数重写来减少这部分成本。

2 用人类知识决定哪些位置不用算

最直觉的加速方式是：有些位置明显不重要，干脆不要算。也就是说，我们先用人类对任务的理解设计一个 mask，只保留某些 query-key 对，其他位置直接设为零。这样做不需要模型先判断哪些位置重要，规则在训练前就固定。

2.1 Local Attention 与 Truncated Attention

Local attention 的核心假设是：很多任务中，一个位置最需要看的只是附近的位置。举例来说，语音或影像中相邻时间点、相邻 patch 的相关性往往较强；若只让每个位置看左右固定窗口，就能把 dense attention matrix 变成靠近对角线的带状矩阵。

Local Attention / Truncated Attention

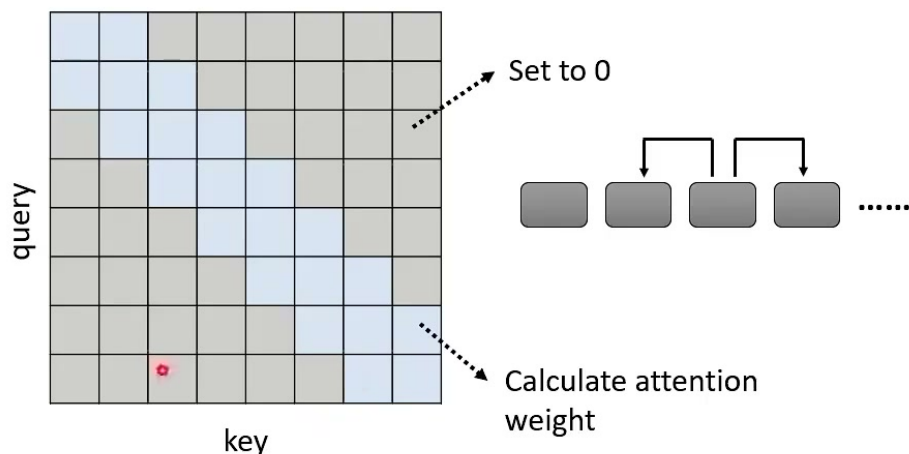


图 2: Local attention 只计算对角线附近的蓝色区域, 灰色位置直接设为零; 这样可以减少 attention matrix 中需要计算的位置数。⁴

若窗口大小为 w , 每个 query 只看大约 w 个 key, 复杂度可由 $O(N^2)$ 降为 $O(Nw)$ 。当 w 远小于 N 时, 这个降幅很可观。

但 local attention 的缺点也很清楚: 它牺牲了 self-attention 最吸引人的性质, 也就是任意两个位置可以一步交互。若某个位置需要远距离信息, local attention 必须靠堆叠很多层, 让信息一层一层传过来。这样它会变得有点像 CNN: 局部感受野便宜, 但长距离依赖要靠层数累积。

Local attention 的代价

Local attention 不是免费午餐。它减少计算, 是因为它明确禁止某些远距离连接。若任务强烈依赖远距离关系, 例如长文本的跨段指代、长音频中的全局语义、图像中相隔很远区域的对应关系, 固定局部窗口可能会漏掉重要信息。

2.2 Global Attention: 给少数 token 全局权限

为了弥补 local attention 的局部性, 可以加入少数 global token。做法是指定一些特殊位置, 让它们可以 attend 到所有 token, 也允许所有 token attend 到它们。这样全局信息可以先汇聚到 special token, 再由 special token 分发给其他位置。

⁴视频画面时间区间: 约 08:17–09:11; 字幕附近说明灰色位置「直接设为零」、只计算蓝色部分, 并指出 local/truncated attention 只能看小范围资讯。

Global Attention

Add special token into original sequence

- Attend to every token → collect global information
- Attended by every token → it knows global information



图 3: Global attention 会让特殊 token 读取所有位置，也让所有位置读取特殊 token；这些 token 像全局信息交换站。⁶

在文本任务中，BERT 的 CLS token 就很容易被想成一种全局汇聚点。对于分类任务，CLS 可以收集整句信息；对于长序列模型，少数 global token 可以在固定稀疏图上建立全局捷径。

不过，global token 也不是万能的。若所有长距离信息都必须经过少数几个节点，模型容量可能受到限制；如果 global token 选择不好，重要信息未必能有效通过它们传递。因此许多模型会把 local、global、random 或 dilated pattern 混合使用。

2.3 Longformer 与 BigBird：固定稀疏图的组合

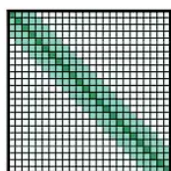
Longformer 和 BigBird 展示了固定稀疏 pattern 可以有很多设计。Longformer 常见元素包括 sliding window attention、dilated sliding window，以及 global sliding window。BigBird 则在 local 和 global 之外加入 random attention，也就是随机允许一些远距离位置彼此连接。

⁶视频画面时间区间：约 12:31–13:28；字幕附近说明 special token 会 attend 到所有 token，也会被所有 token attend，并提到可直接把序列开头 token 当成 special token。

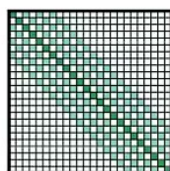
Many Different Choices ...

- Longformer

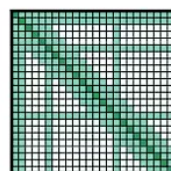
<https://arxiv.org/abs/2004.05150>



(b) Sliding window attention



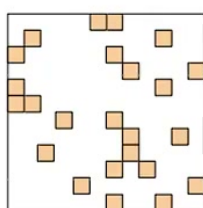
(c) Dilated sliding window



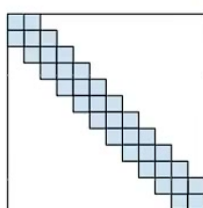
(d) Global+sliding window

- Big Bird

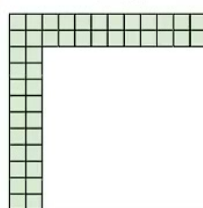
<https://arxiv.org/abs/2007.14062>



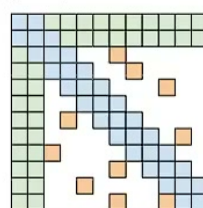
(a) Random attention



(b) Window attention



(c) Global Attention



(d) BIGBIRD

图 4: Longformer 与 BigBird 使用不同的稀疏 attention pattern: 局部窗口、扩张窗口、全局连接、随机连接可以组合起来, 形成比纯 local 更灵活的稀疏图。⁸

随机连接听起来随意, 但它有一个重要作用: 在稀疏图中缩短任意两个位置之间的路径。纯 local attention 中, 信息从序列左端传到右端要经过很多层; 加入少量随机边后, 图的直径可能显著下降。换句话说, random attention 不一定是为了精确建模某个语义关系, 而是为了让信息传播更容易。

固定稀疏 pattern 的优缺点

固定 pattern 的优点是简单、可控、容易实现高效 kernel; 缺点是它把「哪些位置重要」写死了。若任务中的关键依赖关系不符合人类设计的 pattern, 模型可能会为了省计算而错过真正重要的连接。

2.4 本章小结

用人类知识设计稀疏 pattern, 是改造 self-attention 的第一条路。Local attention 用局部性换速度, global token 提供全局汇聚, Longformer 和 BigBird 则把多种连接方式组合起来。它们共同的特点是: 哪些位置要算, 在模型看见具体输入前就已经决定。

⁸视频画面时间区间: 约 18:01–18:40; 字幕附近说明 BigBird 包含 Longformer 的 local/global attention, 并额外加入 random attention, 论文中还有理论推导。

3 让模型判断关键位置

固定稀疏 pattern 很便宜，但未必最适合每个输入。于是第二个想法出现了：能不能让模型或算法根据当前输入判断哪些 attention 值可能大，只计算这些关键位置？

3.1 只保留可能较大的 attention 值

如果某些 attention weight 本来就接近零，那么把它们直接设为零，对最终 weighted sum 的影响可能很小。此时加速的关键不是「强行固定某种 pattern」，而是快速估计哪些位置可能重要。

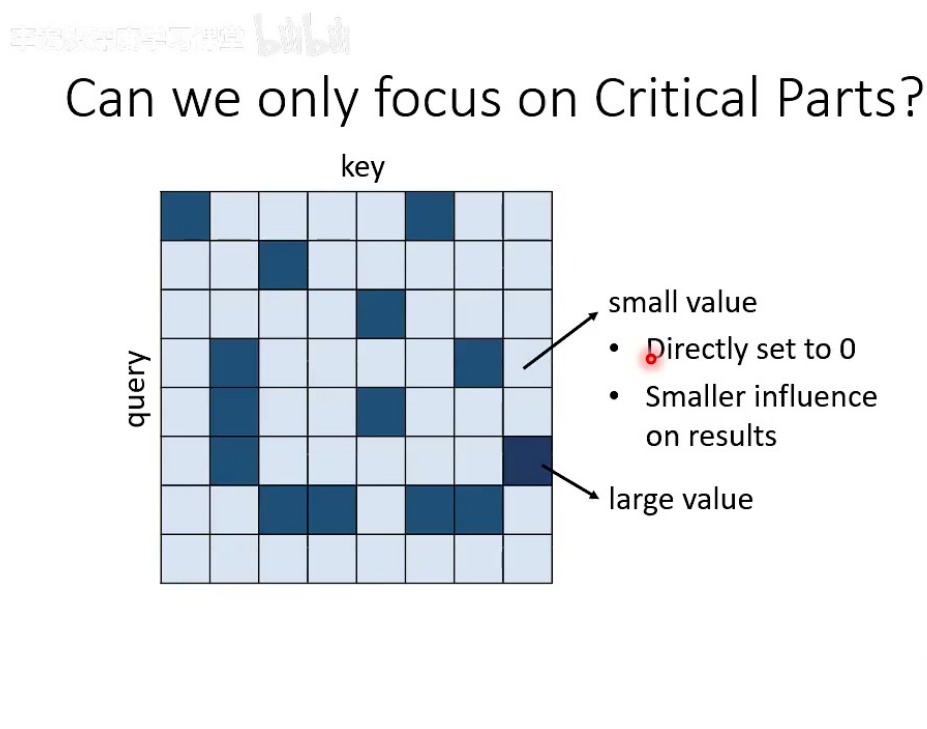


图 5: 只关注 critical parts 的想法：深色块表示可能较大的 attention 值，浅色块可直接设为零，以减少实际计算。¹⁰

这个思路有一个微妙点：标准 attention 是先算出所有 e_{ij} ，再通过 softmax 得到 α_{ij} 。如果你已经把所有 e_{ij} 都算完，再丢掉小值，那计算量并没有省下多少。真正要加速，必须在完整 attention 计算之前，就用更便宜的方法筛出候选位置。

3.2 Reformer 与 Routing Transformer：用 clustering 筛连接

Reformer 和 Routing Transformer 采用相近思想：先把 query 和 key 按相似性分群，只让同一群内的 query-key 对计算 attention，跨群的连接则设为零。

¹⁰视频画面时间区间：约 19:17–20:27；字幕附近说明若某些 attention weight 很小，可以直接设为零，并追问如何估计哪些位置可能有较大值。

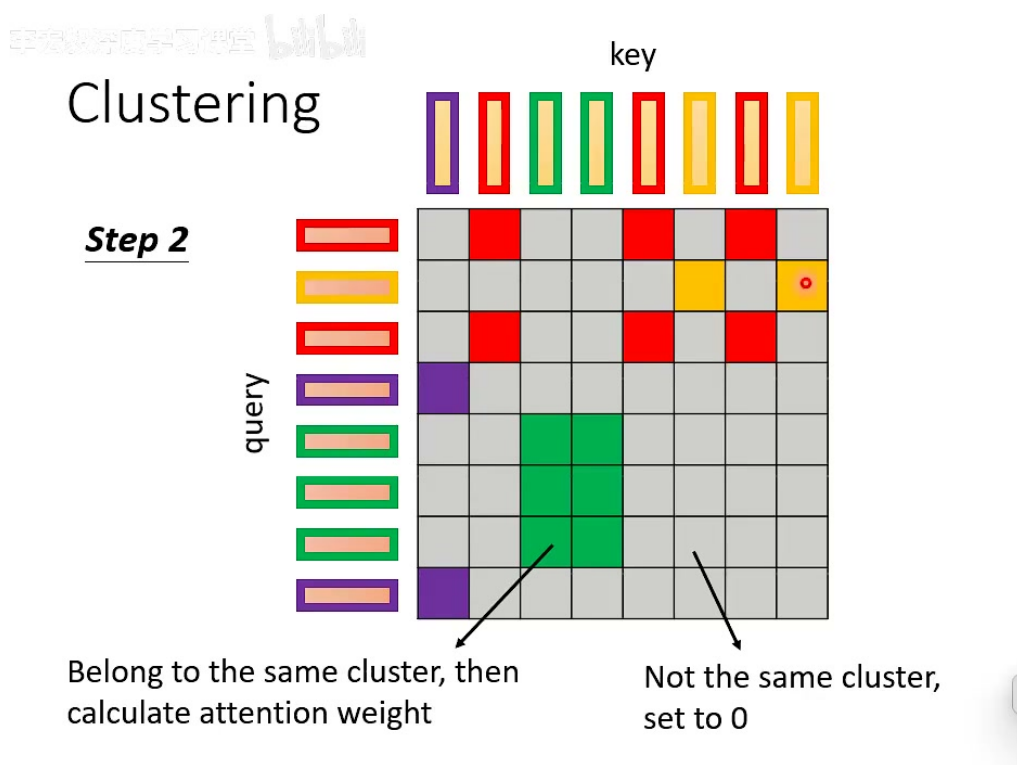


图 6: Clustering 方法会把相近的 query/key 分到同一群；只有同一 cluster 内的位置计算 attention，不同 cluster 的位置直接设为零。¹²

这种方法背后的假设是：若 query 和 key 在表示空间中相距很远，它们的 dot product 往往不会大，最终 attention weight 可能很小。与其对所有位置精算，不如先用近似、较快的 clustering 找出可能相关的区域。

要注意的是，clustering 本身也有成本。若分群算法也需要 $O(N^2)$ ，那就失去了意义。因此实际方法会使用近似分群、哈希、routing 等技巧，让筛选阶段比完整 attention 更便宜。

3.3 Sinkhorn Sorting Network: 可学习的稀疏 pattern

还有一种更大胆的方法：让另一个网络决定哪些格子要算。Sinkhorn Sorting Network 的图中，输入序列通过一个小网络产生类似 mask 的矩阵；这个 mask 决定 attention matrix 中哪些位置保留，哪些跳过。

¹²视频画面时间区间：约 22:48–23:43；字幕附近说明红色 query 只与同 cluster 的 key 计算 attention，其余位置设为零，因此矩阵中很多位置不需要计算。

Learnable Patterns

Sinkhorn Sorting Network

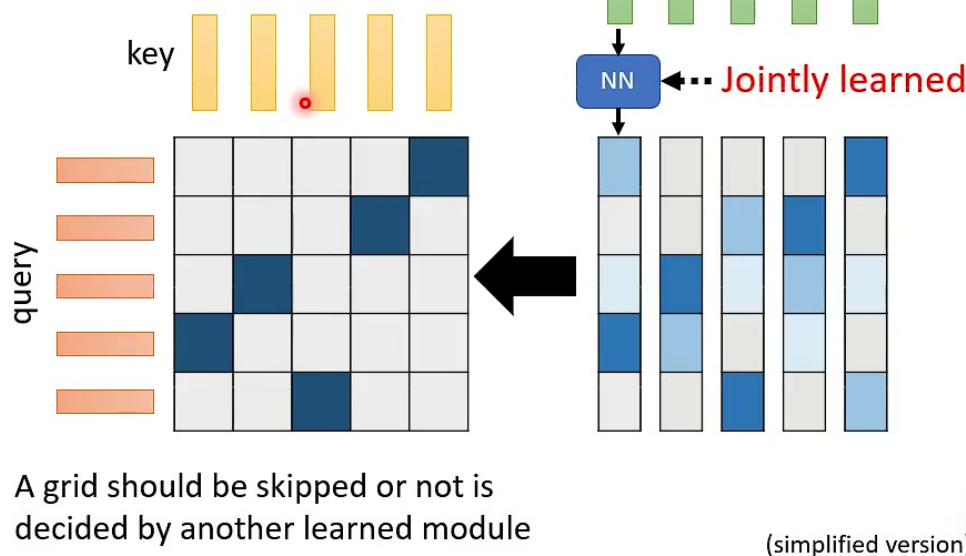


图 7: Sinkhorn Sorting Network 用另一个可学习模块决定哪些 attention 位置要保留；图中的 mask pattern 与主网络一起训练。¹⁴

这里的困难是：是否计算某个格子本质上是 binary decision，但神经网络输出通常是 continuous。若直接把连续值硬切成 0 或 1，就可能造成不可微的问题。Sinkhorn 类方法的重点之一，就是用特殊技巧让「近似排序」或「近似二值 mask」仍可放进可训练系统。

从固定 pattern 到学习 pattern

Local attention 和 BigBird 这类方法把稀疏图写在架构里；Reformer 这类方法用输入相关的 clustering 产生稀疏图；Sinkhorn 进一步把稀疏图本身交给可学习模块决定。越往后，适应性越强，但实现和训练也越复杂。

3.4 本章小结

数据相关稀疏化试图避免固定规则的僵硬性。它的共同目标是用较低成本预测哪些 attention 值值得精算。Reformer/Routing Transformer 通过 clustering 找相关区域；Sinkhorn Sorting Network 则让网络学习稀疏 pattern。关键风险在于：筛选过程本身不能太贵，也不能误删真正重要的连接。

¹⁴ 视频画面时间区间：约 29:46–30:46；字幕附近说明由网络产生较低解析度的 mask，例如先产生 10×10 ，再放大成 100×100 ，并可与整个网络一起反向传播训练。

4 低秩与代表性 Key/Value

前面的稀疏方法仍然保留 $N \times N$ attention matrix 的坐标系，只是不计算某些格子。另一条路线是问：我们真的需要完整的 N 个 key 和 N 个 value 吗？如果 attention matrix 里很多列是重复的、相依的，或可以由少数列线性组合得到，那么也许只需保留少数代表性 key/value。

4.1 Attention Matrix 可能是低秩的

Linformer 的直觉是：实际训练出来的 attention matrix 常常有冗余。若很多 column 之间高度相关，完整 $N \times N$ 矩阵包含的信息并没有看起来那么多。

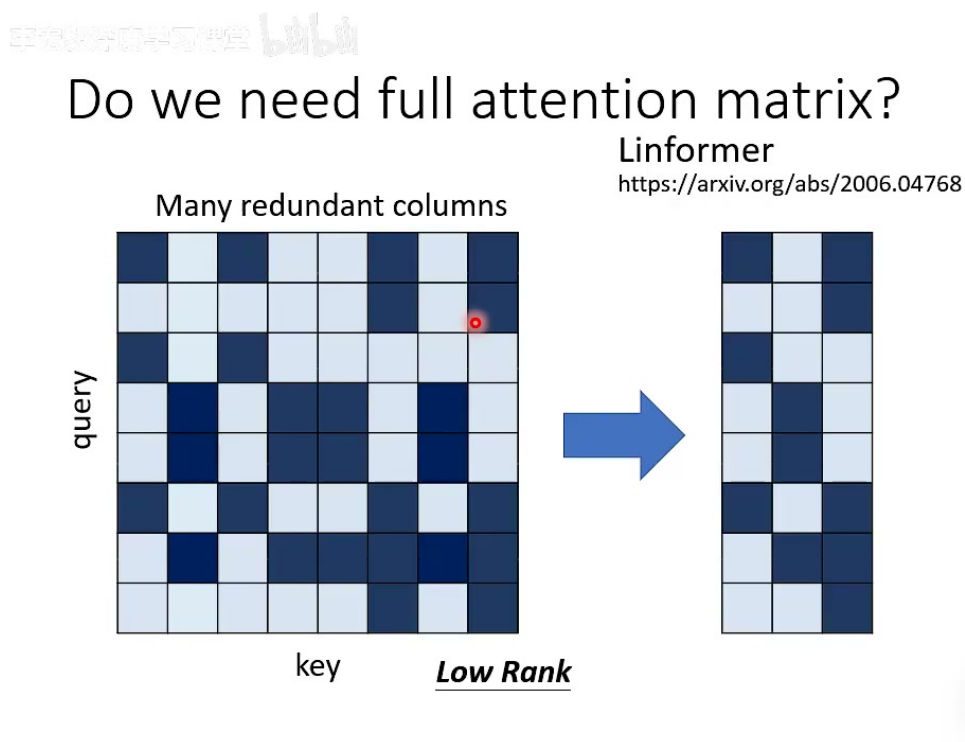


图 8: 若 attention matrix 中许多 column 是冗余的或可由其他 column 线性组合得到，就可以把完整矩阵近似为较低秩的矩阵。¹⁶

低秩观点可以写成矩阵近似。若原 attention matrix 为 $A \in \mathbb{R}^{N \times N}$ ，低秩假设大致认为它可以由较小的中间维度 K 表示：

$$A \approx UV^T, \quad U, V \in \mathbb{R}^{N \times K}, \quad K \ll N.$$

此时存储和计算都可以围绕 K 展开，而不是围绕 N 的平方展开。

4.2 减少 Key 与 Value 的数量

一个直接实现是挑出 K 个代表性 key。原本每个 query 要和 N 个 key 比较；若只和 K 个代表性 key 比较，attention matrix 就从 $N \times N$ 变成 $N \times K$ 。对应地，value 也要压成 K 个代表性

¹⁶视频画面时间区间：约 31:32-32:13；字幕附近说明 attention matrix 可能是 low rank，很多 column 可能是 duplicate 或 linear combination，因此不一定需要完整 $N \times N$ 。

value，因为最后 weighted sum 需要和 key 一一对应的 value。

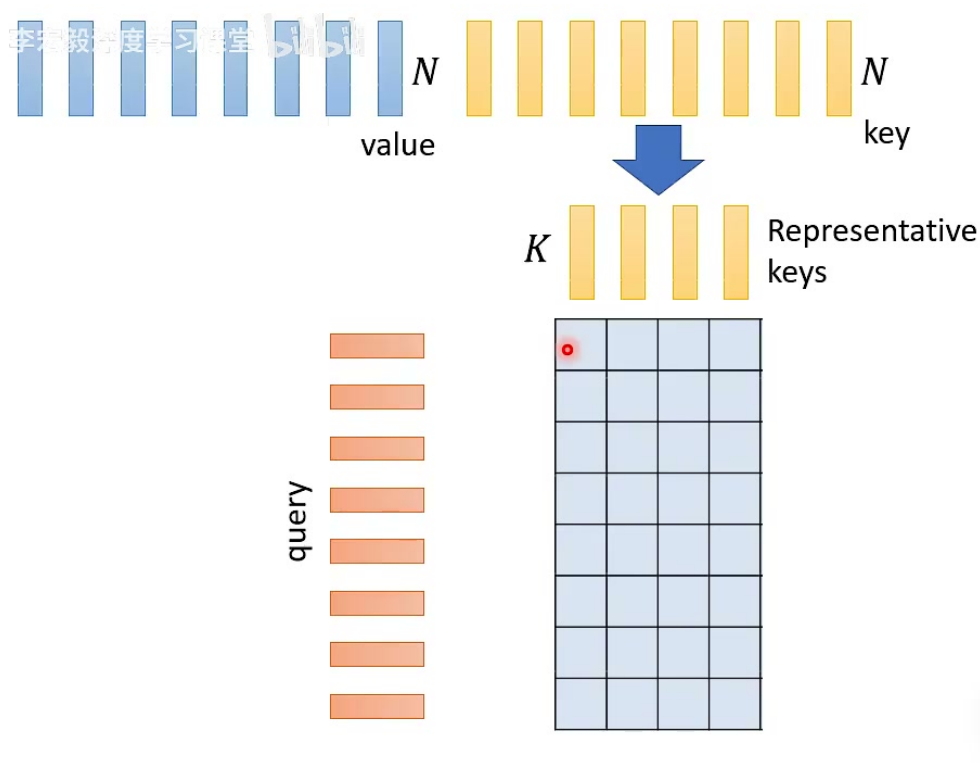


图 9: 只保留 K 个代表性 key/value 后，attention matrix 由 $N \times N$ 变为 $N \times K$ ，每个 query 只需要和代表性 key 比较。¹⁸

这样做的优势是输出长度仍为 N ：每个 query 都还在，所以每个输入位置仍能得到一个输出。它适用于需要对每个位置输出结果的任务，例如逐帧分类、序列标注、语音帧级判断等。

4.3 能不能减少 Query ?

如果减少 query，情况就不一样了。query 的数量决定输出位置数： N 个 query 会产生 N 个输出；若只留下 K 个 query，输出长度就变成 K 。这在某些任务中可以接受，例如整段输入只需要输出一个分类标签；但在每个位置都要预测标签的任务中，减少 query 会直接改变输出形状。

¹⁸视频画面时间区间：约 32:49–33:46；字幕附近说明只挑有代表性的 key，矩阵会「变瘦」，并且 value 也要挑出对应的 K 个代表性向量。

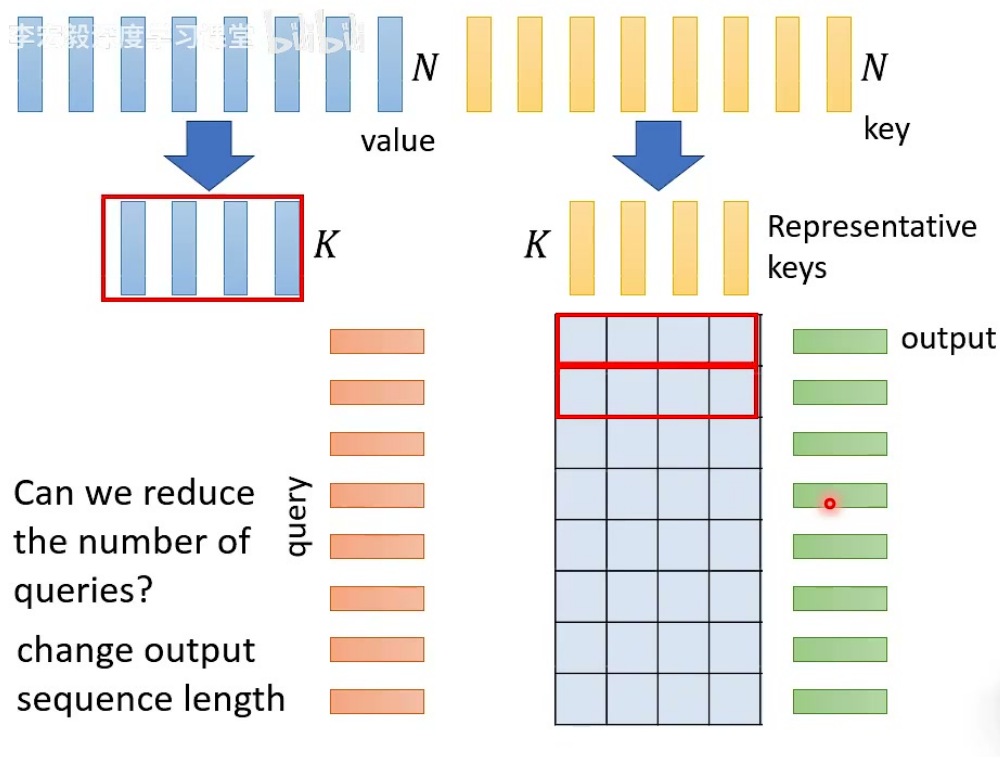


图 10: 减少 query 会改变输出序列长度; 若任务需要每个输入位置都有一个 label, 就不能随便把 query 数量变少。²⁰

这点容易被忽略。减少 key/value 主要是减少每个 query 可看的对象; 减少 query 则改变要输出多少个结果。前者更多是内部计算优化, 后者可能改变任务定义。

4.4 Compressed Attention 与 Linformer

课程中提到两种产生代表性 key 的方式。Compressed Attention 可以用卷积等方法把长序列压成短序列; Linformer 则把 key 矩阵乘上一个 $N \times K$ 的投影矩阵。若 key 矩阵写成 $K_{\text{full}} \in \mathbb{R}^{d \times N}$, Linformer 型投影可写成

$$K_{\text{compressed}} = K_{\text{full}}E, \quad E \in \mathbb{R}^{N \times K},$$

于是 $K_{\text{compressed}} \in \mathbb{R}^{d \times K}$ 。value 也可用类似方式压缩。

²⁰视频画面时间区间: 约 35:19–36:12; 字幕附近用 speaker classification 与逐帧分类对比, 说明有些任务可以减少 query, 有些任务不行。

Reduce Number of Keys

Compressed Attention
<https://arxiv.org/abs/1801.10198>

Linformer
<https://arxiv.org/abs/2006.04768>

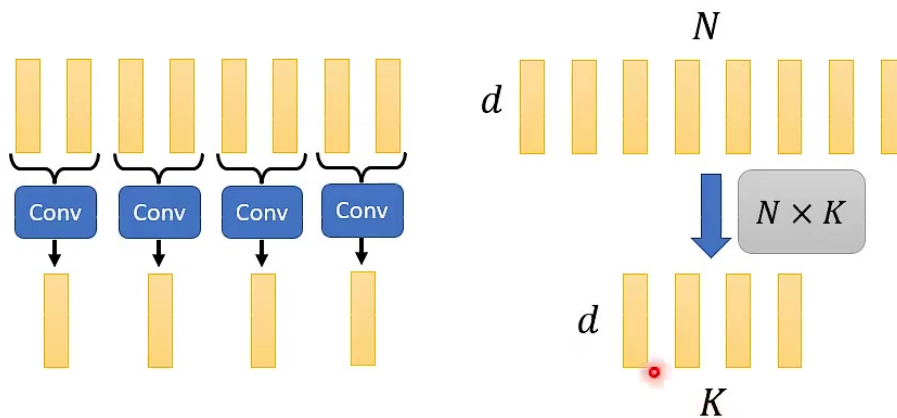


图 11: Compressed Attention 可用卷积压缩序列, Linformer 可用 $N \times K$ 投影矩阵把 $d \times N$ 的 key 压成 $d \times K$ 。²²

Key/Value 压缩的关键问题

压缩是否有效, 取决于代表性 key/value 是否保留了任务需要的信息。若 K 太小, 模型会漏信息; 若 K 太大, 节省的计算有限。低秩方法通常需要在速度、记忆体和近似误差之间取平衡。

4.5 本章小结

低秩和代表性 key/value 方法不是问「哪些格子不算」, 而是问「矩阵是否本来就可以更小」。减少 key/value 能降低 attention matrix 的宽度, 同时保持输出长度; 减少 query 则会改变输出长度, 需要看任务是否允许。Linformer 的核心是用线性投影把 N 个 key/value 压成 K 个代表性向量。

5 把 Attention 看成三矩阵乘法

接下来课程进入更代数化的路线。暂时忽略 softmax, self-attention 可以被看成三个矩阵相乘:

$$O \approx VK^TQ.$$

这里采用课程投影片中的方向, $Q, K \in \mathbb{R}^{d \times N}$, $V \in \mathbb{R}^{d' \times N}$, 因此 $K^TQ \in \mathbb{R}^{N \times N}$, 再乘上 V 得到 $O \in \mathbb{R}^{d' \times N}$ 。

²²视频画面时间区间: 约 36:44-37:40; 字幕附近说明可用 CNN 把长序列变短, 或把 $d \times N$ key 矩阵乘上 $N \times K$ 矩阵得到 $d \times K$ 。

Attention Mechanism is three-matrix Multiplication

Review

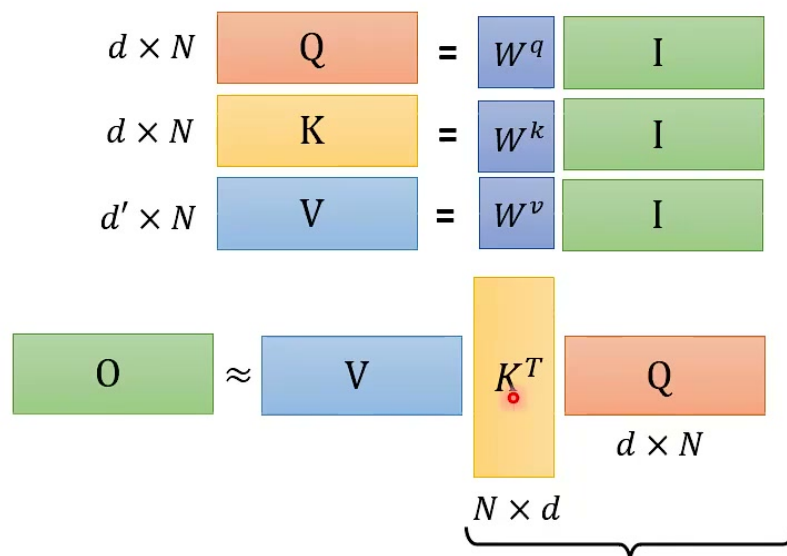


图 12: 忽略 softmax 时, attention 可被看成 V, K^T, Q 三个矩阵相乘; 标准做法会先形成 $K^T Q$ 这个 $N \times N$ 中间矩阵。²⁴

5.1 乘法顺序为什么重要

矩阵乘法满足结合律:

$$V(K^T Q) = (VK^T)Q.$$

数学上结果相同, 但计算成本不同。若先算 $K^T Q$, 中间会产生 $N \times N$ 矩阵; 若先算 VK^T , 中间矩阵大小是 $d' \times d$ 。当 N 很大、 d, d' 相对较小时, 后者显然便宜。

粗略比较如下。若先算 $K^T Q$:

$$K^T Q : O(N^2 d), \quad V(K^T Q) : O(d' N^2).$$

若改成先算 VK^T :

$$VK^T : O(d' d N), \quad (VK^T)Q : O(d' d N).$$

所以后者大约随 N 线性成长, 而不是随 N^2 成长。

为什么不能直接宣布问题解决

标准 attention 中间还有 softmax。Softmax 是对每个 query 的所有 key 分数做归一化, 它依赖完整的一行 $K^T Q$ 。因此只靠矩阵结合律, 只能解释「没有 softmax 时」为什么可以加速。若要处理真实 attention, 还必须说明 softmax 或类似归一化如何被改写或近似。

²⁴视频画面时间区间: 约 40:45–41:47; 字幕附近说明真正做的是 V, K^T, Q 三个矩阵相乘, 先把 K^T 和 Q 乘起来, 再乘 V 得到 O 。

5.2 本章小结

把 attention 写成三矩阵乘法后，瓶颈变得更清楚：标准顺序会显式生成 $N \times N$ 的 $K^\top Q$ ；若能合法地改为先算 $V K^\top$ ，复杂度就可能从平方级降到线性级。接下来的 linear attention 就是在尝试把 softmax 也纳入这种重排。

6 Linear Attention: 把 Softmax 拆成特征映射

课程接下来把 softmax 放回公式中。对第一个位置的输出 b^1 ，标准 attention 可以写成

$$b^1 = \sum_{i=1}^N \frac{\exp(q^1 \cdot k^i)}{\sum_{j=1}^N \exp(q^1 \cdot k^j)} v^i.$$

这个式子显示， b^1 是所有 value 的 weighted sum，权重来自 q^1 与每个 k^i 的 softmax。

$$b^1 = \sum_{i=1}^N \alpha'_{1,i} v^i = \sum_{i=1}^N \frac{\exp(q^1 \cdot k^i)}{\sum_{j=1}^N \exp(q^1 \cdot k^j)} v^i$$

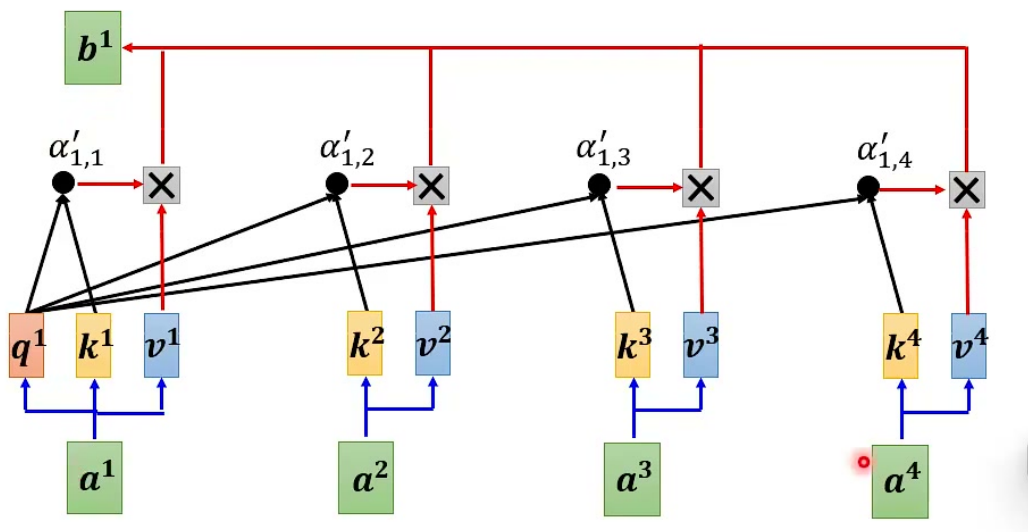


图 13: 标准 attention 中， b^1 由 q^1 对所有 key 的 softmax 权重加权所有 value 得到。²⁵

6.1 Kernel 化的关键假设

Linear attention 的关键一步，是希望把

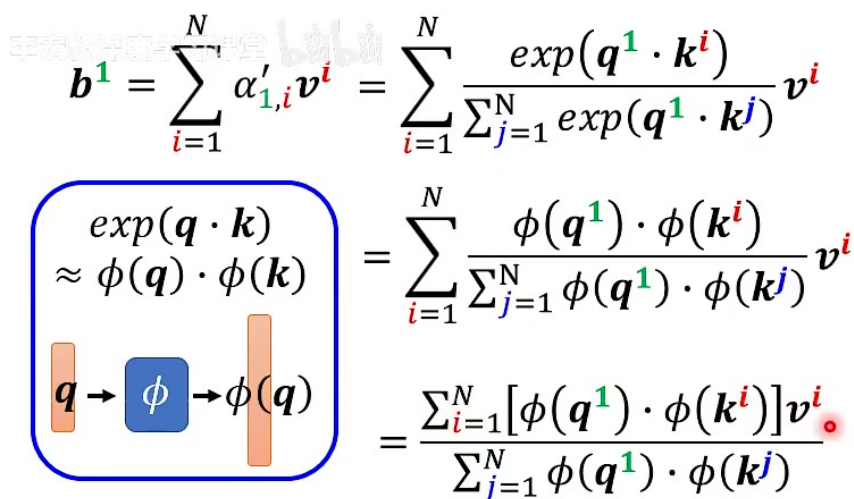
$$\exp(q \cdot k)$$

²⁵ 视频画面时间区间：约 46:44–47:43；字幕附近说明若要计算 b^1 ，就把 q^1 与所有 key 算 attention，再对 value 做 weighted sum。

改写或近似为两个特征向量的内积：

$$\exp(q \cdot k) \approx \phi(q)^\top \phi(k).$$

这里 $\phi(\cdot)$ 是某种 feature map。不同论文对 ϕ 的选择不同：有的使用确定性映射，有的使用随机特征，有的追求无偏估计，有的追求数值稳定。



$$b^1 = \sum_{i=1}^N \alpha'_{1,i} v^i = \sum_{i=1}^N \frac{\exp(q^1 \cdot k^i)}{\sum_{j=1}^N \exp(q^1 \cdot k^j)} v^i$$

$$\approx \sum_{i=1}^N \frac{\phi(q^1) \cdot \phi(k^i)}{\sum_{j=1}^N \phi(q^1) \cdot \phi(k^j)} v^i$$

$$= \frac{\sum_{i=1}^N [\phi(q^1) \cdot \phi(k^i)] v^i}{\sum_{j=1}^N \phi(q^1) \cdot \phi(k^j)}$$

图 14: 若能把 $\exp(q \cdot k)$ 近似为 $\phi(q)^\top \phi(k)$ ，就可以把 softmax attention 改写成 feature map 之间的内积形式。²⁷

将近似代入 b^1 ：

$$b^1 = \frac{\sum_{i=1}^N \phi(q^1)^\top \phi(k^i) v^i}{\sum_{j=1}^N \phi(q^1)^\top \phi(k^j)}.$$

因为 $\phi(q^1)$ 与求和索引 i, j 无关，可以把它从求和中提出或重排。分母可写成

$$\phi(q^1)^\top \left(\sum_{j=1}^N \phi(k^j) \right).$$

分子则可以看成先把每个 $\phi(k^i)$ 与 v^i 做外积，再对 i 求和，最后由 $\phi(q^1)$ 选择组合。

6.2 分子重排：从逐位置加权到全局模板

投影片把分子展开，说明它不是魔法，而是普通代数整理。假设 $\phi(q)$ 和 $\phi(k)$ 的维度为 M 。那么

$$\phi(q^1)^\top \phi(k^i) = \sum_{m=1}^M \phi_m(q^1) \phi_m(k^i).$$

²⁷ 视频画面时间区间：约 50:00–50:57；字幕附近开始整理分子分母，并把 $\exp(q \cdot k)$ 替换成 $\phi(q)$ 与 $\phi(k)$ 的内积。

代回分子：

$$\sum_{i=1}^N \left(\sum_{m=1}^M \phi_m(q^1) \phi_m(k^i) \right) v^i = \sum_{m=1}^M \phi_m(q^1) \left(\sum_{i=1}^N \phi_m(k^i) v^i \right).$$

括号中的项

$$T_m = \sum_{i=1}^N \phi_m(k^i) v^i$$

只和所有 key/value 有关，不和当前 query 是第几个位置有关。

$$\begin{aligned} b^1 &= \sum_{i=1}^N \alpha'_{1,i} v^i = \frac{\sum_{i=1}^N [\phi(q^1) \cdot \phi(k^i)] v^i}{\phi(q^1) \cdot \sum_{j=1}^N \phi(k^j)} \\ &\quad \sum_{i=1}^N [\phi(q^1) \cdot \phi(k^i)] v^i \quad \phi(q^1) = \begin{bmatrix} q_1^1 \\ q_2^1 \\ \vdots \end{bmatrix} \quad \phi(k^1) = \begin{bmatrix} k_1^1 \\ k_2^1 \\ \vdots \end{bmatrix} \\ &= [\phi(q^1) \cdot \phi(k^1)] v^1 + [\phi(q^1) \cdot \phi(k^2)] v^2 + \dots \\ &= (q_1^1 k_1^1 + q_2^1 k_2^1 + \dots) v^1 + (q_1^1 k_1^2 + q_2^1 k_2^2 + \dots) v^2 + \dots \\ &= \underline{q_1^1 k_1^1 v^1} + \underline{q_2^1 k_2^1 v^1} + \dots + \underline{q_1^1 k_1^2 v^2} + \underline{q_2^1 k_2^2 v^2} + \dots + \dots \\ &= q_1^1 (k_1^1 v^1 + k_1^2 v^2 + \dots) + q_2^1 (k_2^1 v^1 + k_2^2 v^2 + \dots) \end{aligned}$$

图 15: 把分子展开后，可以把乘上同一维 query feature 的项收集在一起；这些括号里的向量只由 key/value 决定，可被多个 query 共用。²⁹

从这个角度看，linear attention 先从整个序列中算出一组全局模板 $T_1, \dots, T_M$ ，每个位置的 query 再用自己的 $\phi(q^t)$ 去组合这些模板。分母同理，也可以先算出

$$z = \sum_{j=1}^N \phi(k^j),$$

再用 $\phi(q^t)^\top z$ 做归一化。

6.3 为什么它是线性的

计算 T_m 与 z 都只需要扫过序列一次。对所有位置来说，key/value 产生的这些模板可以共用。计算 b^1 后，计算 b^2 不需要重新求一次所有模板，只需要换成 $\phi(q^2)$ 去组合。

²⁹ 视频画面时间区间：约 53:14–54:16；字幕附近说明把乘上 q_1^1 、 q_2^1 的项分别集合，得到若干组 value 的 linear combination。

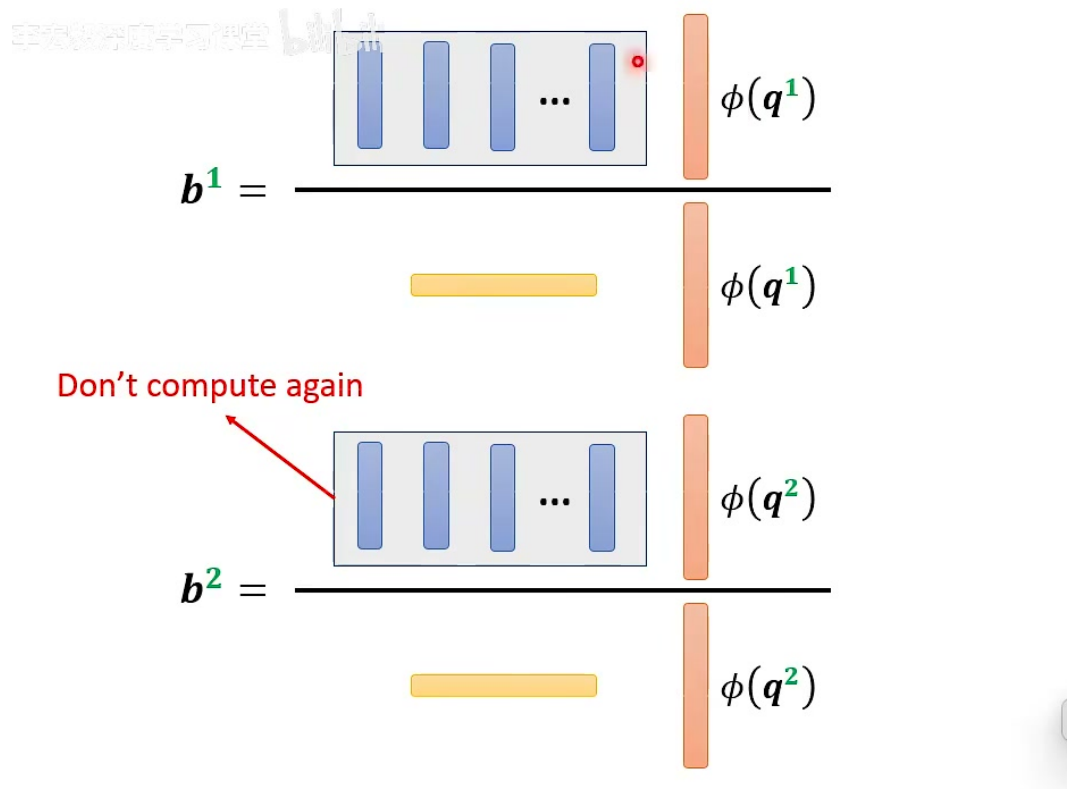


图 16: Linear attention 中，由 key/value 产生的蓝色向量和黄色归一化项只需计算一次；不同位置只要换不同的 $\phi(q)$ 去组合它们。³¹

若 feature map 维度 M 视为常数或远小于 N ，整个过程复杂度大约是

$$O(NMd') + O(NM),$$

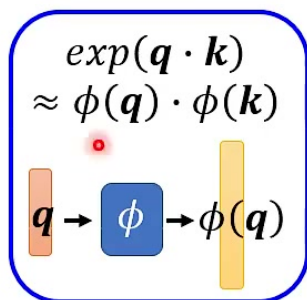
随序列长度 N 线性成长。代价是：我们不再显式得到完整 attention matrix，也较难像标准 attention 那样直接解释「某个 token attend 到哪个 token」。

6.4 不同论文如何实现

课程列出几种代表做法：Efficient Attention、Linear Transformer、Random Feature Attention、Performer。它们都围绕类似思想，但对 ϕ 的构造、数值稳定、归一化和随机近似有不同处理。

³¹视频画面时间区间：约 57:57–58:39；字幕附近说明计算 b^1 时得到的蓝色 factor 和黄色 factor 在计算 b^2 时不需要重算，只需换成 $\phi(q^2)$ 。

Realization



- Efficient attention

<https://arxiv.org/pdf/1812.01243.pdf>

- Linear Transformer

<https://linear-transformers.com/>

- Random Feature Attention

<https://arxiv.org/pdf/2103.02143.pdf>

- Performer

<https://arxiv.org/pdf/2009.14794.pdf>

图 17: Efficient Attention、Linear Transformer、Random Feature Attention、Performer 都可被放在线性化 attention 的实现族中；差别主要在 feature map 与近似方式。³³

Linear attention 的理解边界

Linear attention 不是把标准 softmax attention 精确免费化。它通常依赖 kernel 近似、feature map 选择或归一化改写。速度变快后，可能牺牲一部分表达能力、训练稳定性或可解释性。实际是否更好，要看任务、序列长度、硬件实现和数值细节。

6.5 本章小结

Linear attention 的核心是把 $\exp(q \cdot k)$ 改写为 $\phi(q)^\top \phi(k)$ ，再利用代数重排，让 key/value 产生的全局模板被所有 query 共用。它绕开显式 $N \times N$ attention matrix，使复杂度有机会随 N 线性成长；但这个收益来自近似和公式改变，而不是无条件等价于标准 attention。

7 还需要用 Q 和 K 算 Attention 吗

前面的方法都仍然保留一个共同设定：attention weight 来自 query 与 key 的关系。最后课程提出更激进的问题：我们真的需要 Q 和 K 吗？

³³视频画面时间区间：约 64:15–64:30；字幕附近说明不同论文用不同方式拆解 $\exp(q \cdot k)$ ，并列出了 Efficient Attention、Linear Transformer、Random Feature Attention、Performer。

7.1 Synthesizer: Attention Matrix 可以是参数

Synthesizer 的想法很反常识。它不一定用 q 和 k 的内积产生 attention matrix。某些版本会让输入经过网络产生矩阵；还有更极端的版本，直接把 attention matrix 视为网络参数的一部分。也就是说，矩阵中的 $N \times N$ 个数本身就是可训练参数。

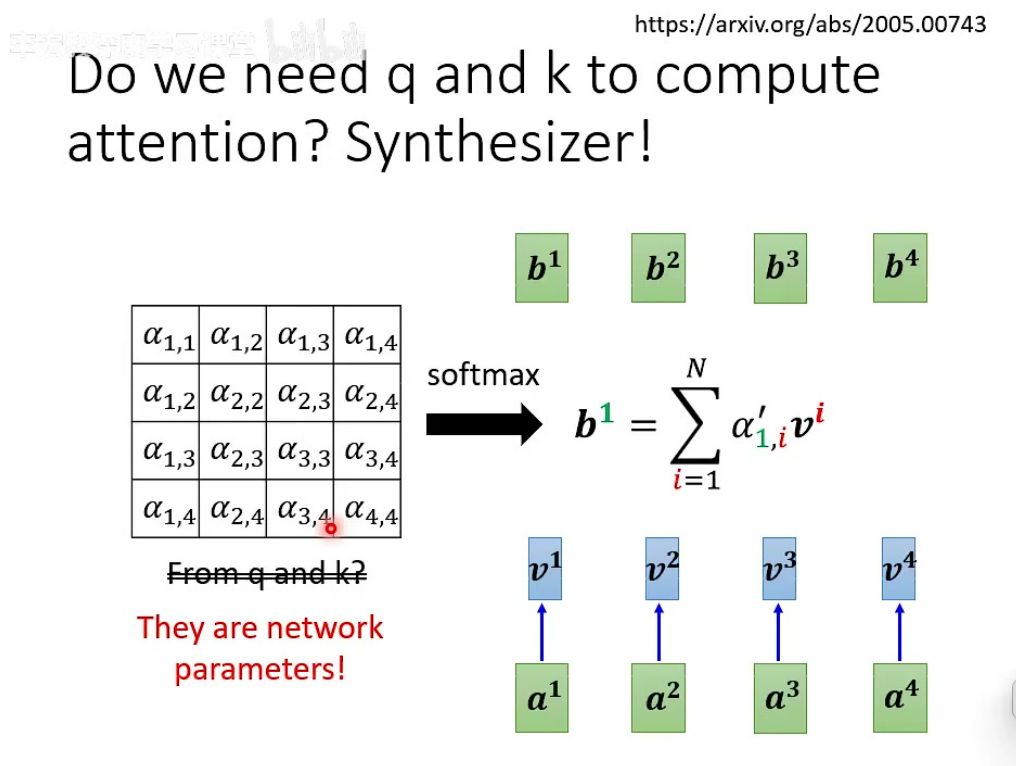


图 18: Synthesizer 可以不由 Q 和 K 计算 attention matrix，而是把矩阵看成网络参数，直接学习一组 mixing weights。³⁵

这个方法的启示是：self-attention 的本质也许不只是「相似度搜索」，还可以被理解为一种 token mixing。若任务只需要在位置之间混合信息，那么混合矩阵未必一定要来自 query-key dot product。

当然，这也带来明显限制。若 attention matrix 是固定参数，它对输入内容的适应性较弱；序列长度变化时也需要额外处理。它更像是在提醒我们：Transformer 的成功不一定只来自某一个固定解释，许多机制都可能在不同层面贡献效果。

7.2 Attention-free 与 MLP Mixing

课程也提到 FNet、Pay Attention to MLPs、MLP-Mixer 等 attention-free 方向。这些方法尝试不用传统 attention，而用 Fourier transform、MLP 或其他 mixing 机制在 token 间传递信息。

它们和 Synthesizer 有相似精神：如果目标是让不同位置交换信息，那么并非只有 $QK^\top$ 一条路。不同 mixing 机制在速度、归纳偏置、硬件友好性和表达能力上各有取舍。

³⁵ 视频画面时间区间：约 65:44–66:44；字幕附近说明 Synthesizer 中 attention matrix 不是由 Q 和 K 产生，甚至可以直接作为网络中的 $N \times N$ 个参数。

7.3 本章小结

Synthesizer 与 attention-free 方法把问题推到更高层：我们到底需要的是 attention 这个公式，还是需要 token 间的信息混合能力？它们不一定取代标准 attention，但能帮助我们理解 Transformer 架构中哪些部分是核心、哪些部分可以被替换。

8 总结与延伸

本讲最后用 Long Range Arena 的结果比较不同方法。图中纵轴代表效果分数，横轴代表速度，圆圈大小代表记忆体使用。可以看到，方法之间没有单一赢家：有些方法很快但效果掉得多，有些方法效果接近但速度优势有限，有些方法记忆体较省但实现复杂。



Summary

- Human knowledge
 - Local Attention, Big Bird
- Clustering
 - Reformer
- Learnable Pattern
 - Sinkhorn
- Representative key
 - Linformer
- k,q first $\rightarrow$ v,k first
 - Linear Transformer, Performer
- New framework
 - Synthesizer

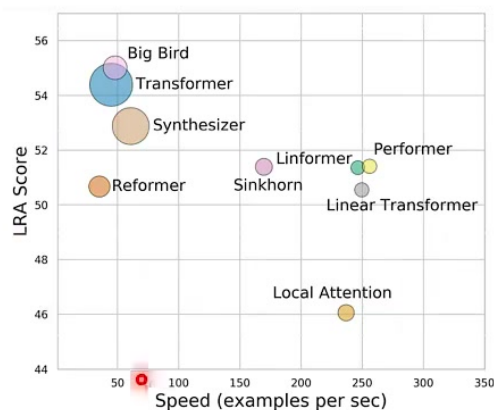


图 19: Long Range Arena 比较了多种高效 self-attention 方法：横轴越右速度越快，纵轴越高效果越好，圆圈越大表示记忆体使用越多。³⁷

8.1 一张方法地图

可以把本讲方法整理为下表。

³⁷ 视频画面时间区间：约 68:01–70:16；字幕附近说明 LRA score 越高代表表现越好，横轴代表速度，圆圈大小代表 memory，并比较 Transformer、Local Attention、Sinkhorn、Performer 等方法。

路线	代表方法	核心取舍
人类设计稀疏图	Local Attention、Longformer、BigBird	简单高效，但连接 pattern 固定，可能漏掉输入相关的重要依赖。
数据相关稀疏化	Reformer、Routing Transformer	根据 query/key 相似性筛选连接，适应性更强，但筛选过程要足够便宜。
可学习 pattern	Sinkhorn Sorting Network	让网络学习 mask，表达更灵活，但训练和可微近似更复杂。
低秩压缩	Linformer、Compressed Attention	减少 key/value 数量，保持输出长度，但依赖低秩或代表性假设。
线性化 attention	Linear Transformer、Performer、Efficient Attention	用 feature map 和代数重排绕开 N^2 ，但通常涉及近似与数值稳定问题。
新 mixing 框架	Synthesizer、FNet、MLP-Mixer	不再执着于 $QK^\top$ ，强调 token mixing，但归纳偏置和泛化表现要重新评估。

表 1: 本讲 self-attention 变型的主要路线。

8.2 理解这些方法时最重要的三个问题

第一，**它省掉了什么**？有的方法省掉 attention matrix 中的格子，有的方法省掉 key/value，有的方法省掉显式矩阵，有的方法省掉 query-key 相似度本身。不同「省法」会带来不同副作用。

第二，**它依赖什么假设**？Local attention 依赖局部性，BigBird 依赖稀疏图仍能保留传播路径，Reformer 依赖相似向量应在同一 cluster，Linformer 依赖低秩，linear attention 依赖 kernel 近似或 feature map 设计。若假设和任务不匹配，速度提升可能换来明显性能损失。

第三，**它在真实系统中是否真的快**？理论复杂度只是第一步。GPU/TPU 上的矩阵乘法、稀疏 kernel、memory bandwidth、batch size、序列长度、实现成熟度都会影响最终速度。某些方法理论上更低阶，但如果实现无法充分利用硬件，实际速度未必占优。

课程给出的总判断

Efficient attention 不是一个单一技巧，而是一组围绕长序列瓶颈展开的设计空间。原始 Transformer 很强，但 N^2 成本迫使我们思考：哪些连接必须精算，哪些信息可以压缩，哪些公式可以重写，哪些架构假设可以被替换。

8.3 延伸阅读方向

若继续深入，可以按三条线阅读。第一条线是长上下文 Transformer，重点看 Longformer、BigBird、Reformer、Performer 等模型如何在真实长序列任务中取舍。第二条线是 kernelized attention，关注 $\phi(q)^\top \phi(k)$ 如何近似 softmax kernel，以及随机特征如何控制误差和稳定性。第三条线是 token mixing 架构，比较 attention、MLP、Fourier transform、卷积和状态空间模型在序列建模中的角色。

本讲最值得带走的不是某个方法名称，而是一种审题方式：当一个模型模块太贵时，先找出真

正贵在哪里，再问它是否必须完整计算。Self-attention 的瓶颈来自 $N \times N$ 两两交互，而本讲所有方法都在用不同方式回答「哪些交互可以省、压、近似或替换」。